Week 19.1

2303A51018

Batch-28

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

Instructions:

• Prompt AI to convert a Python class with attributes and methods into equivalent Java code.

• Verify that constructors, method signatures, and data types are properly handled.

• Run both versions (Python & Java) to compare expected outputs.

Expected Output:

• Equivalent working Java class translated from Python.

Code

AI Lab 19.1

File Edit View Insert Runtime Tools Help

ommands + Code + Text Run all

[3]
0s

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display(self):
        print("Employee Name:", self.name)
        print("Salary:", self.salary)

# Create object
e1 = Employee("Sanjana", 50000)
e1.display()
```

```
Employee Name: Sanjana
Salary: 50000
```

AI Lab 19.1

File Edit View Insert Runtime Tools Help

ommands + Code + Text Run all

[1]
0s

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)

# Create object
s1 = Student("Sanjana", 20)
s1.display()
```

```
Name: Sanjana
Age: 20
```

Step 3: Verification

✓ Constructor Handling

- Python → `__init__(self, name, salary)`
- Java → `Employee(String name, int salary)` ✅

✓ Method Signature

- Python → `def display(self)`
- Java → `void display()` ✅

✓ Data Types

- Python → No type needed
- Java → `String`, `int` explicitly defined ✅

Justification

The Python class was successfully converted into an equivalent Java class using AI.
👉 The constructor in Python (`__init__`) was correctly mapped to a Java constructor.
👉 The methods in Python were converted into Java methods with proper syntax and return types.
👉 Data types, which are not required in Python, were explicitly defined in Java (such as `String` and `int`).
👉 Both the Python and Java programs were executed, and they produced the same output.

👉 Therefore, the conversion is correct and the functionality of the program is maintained.

Conclusion

The Python class was successfully converted into an equivalent Java class using AI.
👉 Constructors, method definitions, and data types were correctly handled.
👉 Both programs produced the same output, confirming correctness.

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

• Ask AI to translate a recursive C++ factorial function into Python.

• Validate correctness by testing with sample inputs.

• Ensure Python version follows Pythonic conventions.

Expected Output:

• A correct Python implementation equivalent to the original C++ factorial function.

✅ 1. Correct C++ Code (Copy exactly)

Make sure your code looks **exactly like this**:

</> C++

```cpp
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    cout << "Factorial: " << factorial(num);
    return 0;
}
```

[7]
✓ 0s

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = 5
print("Factorial:", factorial(num))
```

```
Factorial: 120
```

Verification

✓ Recursion

- C++ → `factorial(n - 1)`
- Python → `factorial(n - 1)` ✅

✓ Base Condition

- Both use: `n == 0 or n == 1` ✅

✓ Syntax Difference

- C++ uses `{}` and `;`
- Python uses indentation

Pythonic Conventions

✓ No data types required
✓ Clean indentation
✓ Simple function definition using `def`

◇ Final Conclusion (Write this)

👉 The recursive C++ factorial function was successfully converted into Python using AI.
👉 The logic and recursion were preserved correctly.
👉 Both programs produced the same output for the given input.
👉 The Python code follows proper Python syntax and conventions.

Justification

👉 The C++ factorial function was correctly translated into Python with proper recursion and base condition. Both versions produced the same output, confirming correctness.

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

• Provide AI with a SQL query (e.g., SELECT name, salary

FROM employees WHERE salary > 50000).

• Ask AI to generate equivalent Pandas code.

• Test the generated code on a sample DataFrame.

Step 1: SQL Query (Given)

```
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

👉 Meaning:

- Get **name and salary**
- Only if **salary > 50000**

Step 2: Equivalent Pandas Code

```python
import pandas as pd

# Sample DataFrame
data = {
    'name': ['Asha', 'Ravi', 'Sanjana', 'Kiran'],
    'salary': [40000, 60000, 75000, 30000]
}

df = pd.DataFrame(data)

# Pandas Query
result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

```
...        name  salary
    1     Ravi   60000
    2  Sanjana   75000
```

Step 3: Output

```
     name  salary
1     Ravi   60000
2  Sanjana   75000
```

Step 4: Explanation

SQL	Pandas
SELECT name, salary	`[['name','salary']]`
FROM employees	`df`
WHERE salary >	`df['salary'] >`
50000	`50000`

Conclusion

👉 The SQL query was successfully converted into an equivalent Pandas operation.
👉 The filtering condition and column selection were correctly implemented.
👉 The output of the Pandas code matches the SQL query result.

Justification

👉 The SQL query was translated into Pandas using DataFrame filtering and column selection. The result matches the expected output.